



UNIVERSITA' DEGLI STUDI DI
NAPOLI FEDERICO II

Scuola Politecnica e delle Scienze di Base
Corso di Laurea Magistrale in Ingegneria Informatica

Tesi di Laurea Magistrale in Ingegneria Informatica

*Integrazione del Protocollo TEC Tree per la
Sicurezza e Autenticazione della Memoria
Off-Chip con il Protocollo di Coerenza
Tardis-TSO*

Anno Accademico 2025/2026

Relatore
Ch.mo prof. Alessandro Cilardo

Candidato
Fuorto Simone
matr. M63001564

Abstract

Con l'aumentare della gamma di servizi forniti dai sistemi embedded, cresce anche la quantità di dati sensibili che essi manipolano. Di conseguenza, esistono attualmente forti incentivi per gli aggressori che desiderano trarre profitto da attacchi a questi sistemi. Gli attacchi sui bus consentono di recuperare i dati in transito da o verso la memoria, sollevando il problema della riservatezza dei dati. In generale, per garantire la riservatezza vengono implementati schemi di cifratura. Tuttavia, nella maggior parte dei casi, la sola crittografia non è sufficiente quando un avversario è in grado di manipolare i dati senza essere rilevato (vedi *bus probing*). L'integrità della memoria è quindi un obiettivo primario per la protezione di una piattaforma informatica contro attaccanti attivi. L'obiettivo di questo lavoro è l'implementazione di un protocollo in grado di garantire integrità e confidenzialità grazie alla cifratura della memoria *off-chip* basata su *Tamper-Evident Counter Tree* e di valutarne le performance in relazione ai costi di realizzazione introdotti.

Indice

Abstract	i
1 Fondamenti Teorici	1
1.1 Contesto e Motivazioni	1
1.2 Autenticazione della Memoria	3
1.2.1 Autenticazione basata su Contatori	3
1.2.2 Tamper Evident Tree	5
1.3 Il Protocollo di Coerenza Tardis-TSO	6
1.3.1 Il modello TSO (Total Store Order)	7
1.3.2 Il meccanismo dei Timestamp logici	7
1.3.3 Architettura del Protocollo	8
1.3.4 Integrazione dell'Autenticazione e Coerenza . .	9
1.4 Ambiente di Simulazione GEM5	11
1.4.1 Modulo Ruby	11
1.5 Obbiettivi e Workflow della Tesi	12

Capitolo 1

Fondamenti Teorici

1.1 Contesto e Motivazioni

Negli ultimi anni, con la crescente diffusione di applicazioni che gestiscono dati sensibili (dal cloud computing ai dispositivi embedded), la sicurezza dell'hardware è diventata un requisito imprescindibile. Nei modelli di minaccia standard per i sistemi informatici, si assume tipicamente che il perimetro di sicurezza (Trusted Computing Base) sia circoscritto al solo chip del processore. Tutto ciò che risiede all'esterno, compresa la memoria principale off-chip e i bus di interconnessione, è intrinsecamente esposto e vulnerabile. Un attaccante che ha accesso fisico al dispositivo potrebbe leggerne e manipolarne il contenuto. Nell'ambito della sicurezza hardware infatti, i possibili attacchi sono divisi in due categorie:

- **Attacchi Passivi** (Snooping o Bus Probing): l'attaccante si

limita principalmente ad "origliare" il bus leggendo informazioni in chiaro in transito da e verso la memoria, violandone la confidenzialità;

- **Attacchi Attivi:** l'attaccante non si limita a leggere i dati, ma ne altera il contenuto, violandone l'integrità. Tra gli attacchi attivi figurano:
 - **Spoofing:** Iniezione di dati casuali o artefatti al posto di quelli reali.
 - **Splicing (o Relocation):** L'attaccante prende un blocco di dati valido da un indirizzo di memoria e lo copia in un altro indirizzo.
 - **Replay Attack:** L'attaccante registra un blocco di dati valido e lo reinserisce nella memoria in un secondo momento, riportando la memoria a uno stato pregresso o obsoleto.

Sebbene la cifratura di dati in uscita dall'ultimo livello di cache sia sufficiente a garantire la confidenzialità contro attacchi passivi, essa risulta del tutto inefficace contro attacchi attivi. Un soggetto malintenzionato può sfruttare le tre tecniche precedentemente elencate per alterare il normale flusso di esecuzione di un programma e corrompere i dati, senza che i core possano rilevarne l'intrusione. L'unico metodo in grado di garantire confidenzialità ma allo stesso tempo anche in-

tegrità è una verifica della *freshness* del dato ovvero l'autenticazione della memoria.

1.2 Autenticazione della Memoria

Per contrastare gli attacchi attivi descritti in precedenza, l'approccio base prevede l'associazione di un *Message Authentication Code* (MAC). Il MAC è un'etichetta crittografica calcolata a partire dai dati in uscita dalla LLC (Last Level Cache) e da una chiave segreta memorizzata saldamente all'interno del chip. Sebbene l'uso di un MAC legato all'indirizzo fisico di memoria impedisca gli attacchi di Spoofing e Splicing, non è sufficiente a sventare i Replay Attack. Un avversario può infatti intercettare una coppia valida composta da "vecchio dato" e "vecchio MAC", per poi sovrascriverla in memoria in un momento successivo: il processore, ricalcolando il MAC sul vecchio dato, lo troverebbe matematicamente corretto, non accorgendosi della manomissione.

1.2.1 Autenticazione basata su Contatori

Per garantire la freschezza del dato e contrastare eventuali attacchi di replay, la crittografia moderna introduce i contatori o indicatori di versione. A ogni blocco di memoria viene associato un contatore che viene incrementato ad ogni operazione di scrittura. Il MAC viene

quindi calcolato includendo anche questo contatore. Tuttavia, poiché lo spazio on-chip è estremamente limitato, i contatori stessi devono essere memorizzati nella RAM off-chip, esponendo anch'essi al rischio di Replay Attack. La soluzione accademica classica a questo problema è il **Bonsai Merkle Tree (BMT)**. Il BMT organizza i contatori in una struttura ad albero n-ario:

- I contatori dei blocchi dati costituiscono le "foglie" dell'albero.
- Ogni nodo intermedio dell'albero è un MAC che protegge l'integrità dei suoi nodi figli.
- La radice dell'albero (*Root*) è l'unico elemento mantenuto permanentemente al sicuro all'interno del processore (on-chip).

In un BMT, quando si legge un blocco di memoria, il processore deve recuperare il suo contatore e verificare l'intera catena di nodi intermedi fino alla radice. Sebbene tale schema offra le massime garanzie di sicurezza, soffre di un profondo limite prestazionale. La verifica del dato infatti, necessita di numerose letture sequenziali in memoria nonché di onerosi e ripetuti calcoli crittografici, introducendo un overhead di latenza e un consumo di banda spesso inaccettabili per le moderne architetture ad alte prestazioni.

1.2.2 Tamper Evident Tree

Per superare il collo di bottiglia temporale del BMT lo stato dell'arte si è evoluto verso strutture crittografiche alternative ad alte prestazioni, tra cui spicca il **Tamper-Evident Counter Tree (TEC-Tree)**. A differenza dei tradizionali alberi di hash, il TEC-Tree adotta un paradigma basato sulla cifratura a blocchi unita alla tecnica *Block-Level AREA* (Added Redundancy Explicit Authentication). In questa struttura, i nodi intermedi dell'albero non sono costituiti da digest di hash calcolati sui livelli inferiori, bensì da *Counter Chunk* (CC), ovvero blocchi di memoria contenenti esclusivamente contatori di versione. I contatori delle foglie proteggono i blocchi dati effettivi (*Data Chunk*), mentre i contatori dei nodi intermedi proteggono i nodi figli, fino ad arrivare alla radice (*Root Counter*) conservata on-chip in modo sicuro. Per garantire sia l'integrità che la confidenzialità, il TEC-Tree concatena a ogni blocco (Dati o Contatori) un *nonce* univoco. Tale nonce è tipicamente composto dall'indirizzo fisico del blocco stesso e dal valore del suo contatore genitore. L'intero aggregato (Payload | Nonce) viene poi cifrato utilizzando un cifrario a blocchi in modalità ECB (*Electronic Code Book*) e scritto in memoria. La cifratura in modalità ECB opera su blocchi atomici e indipendenti, tale approccio permette di rendere il processo di cifratura dei diversi blocchi un processo parallelo, a patto che i blocchi dell'albero dalla foglia alla radice siano stati già prelevati dalla memoria. L'autenticazione avviene semplicemente

importante precisare che il design originale del protocollo Tardis è stato proposto in letteratura da Yu et al[2], mentre l'implementazione base di Tardis-TSO all'interno del simulatore Gem5, utilizzata come punto di partenza per questo elaborato, è frutto di lavori precedenti[1]. Il contributo originale di questa tesi consisterà, come vedremo nel Capitolo 2, nell'estensione e modifica profonda di tale architettura di base per supportare l'albero crittografico TEC-Tree.

1.3.1 Il modello TSO (Total Store Order)

Nei moderni sistemi multicore, la coerenza della cache rappresenta una sfida significativa sia in termini di prestazione che di traffico sul bus. I protocolli tradizionali come MESI o modelli *directorybased* standard, prevedono molteplici scambi di messaggi di invalidazione e di *acknowledgment* per garantire che i core osservino una memoria coerente. Tardis è un protocollo di coerenza altamente scalabile che elimina del tutto i messaggi di invalidazione. Nella sua evoluzione, **Tardis 2.0** (o TardisTSO), il protocollo è stato riadattato per supportare il modello di consistenza *Total Store Order*, tipico delle moderne architetture x86.

1.3.2 Il meccanismo dei Timestamp logici

A differenza dei protocolli tradizionali, Tardis-TSO gestisce la coerenza attraverso l'utilizzo di **Timestamp Logici**. L'ordinamento delle

operazioni di memoria non è garantito da broadcast di rete, bensì dall'assegnazione di contatori logici che definiscono l'ordine globale degli accessi. Per supportare il modello TSO, il protocollo suddivide il tempo logico in due componenti principali associate al processore:

- **Load Timestamp (lts)**: il timestamp dell'ultima operazione di Load eseguita;
- **Store Timestamp (sts)**: il timestamp dell'ultima operazione di Store eseguita.

Ciascuna linea di cache presenta inoltre due contatori ad essa associati;

- **Write Timestamp (wts)**: indica la versione del blocco;
- **Read Timestamp (rts) o Lease Time** : rappresenta una scadenza , ovvero l' intervallo di tempo logico entro il quale il core può leggere liberamente la linea di cache.

Durante l'esecuzione, il Load Timestamp del processore cresce. Quando tale valore supera il lease time (rts) associato al blocco di cache, la linea viene considerata logicamente scaduta e il processore è costretto a richiederne il rinnovo.

1.3.3 Architettura del Protocollo

L'architettura Tardis-TSO si articola su due entità principali che comunicano in modalità *split-transaction*: le cache private di primo livello

(L1) e la Directory centralizzata, che assume il ruolo di *Timestamp Manager*.

- **Cache L1:** mantengono i blocchi di dati e i relativi timestamp locali (wts e rts). Possono possedere i blocchi in stato condiviso (*Shared*) per la lettura o in stato esclusivo (*Exclusive*) per la scrittura. Quando un blocco scade, la cache L1 invia una richiesta mirata alla Directory per ottenere un prolungamento del *lease time*.
- **Directory (Timestamp Manager):** non memorizza esplicitamente i dati ma tiene traccia dello stato e del proprietario di ogni linea di cache, gestendone i timestamp globali. La Directory risolve i conflitti, instrada i blocchi e fornisce i rinnovi del *lease time*, fungendo da arbitro centrale.

1.3.4 Integrazione dell'Autenticazione e Coerenza

Sebbene il TEC-Tree risolva brillantemente il problema della latenza computazionale parallelizzando le decrittazioni, introduce inevitabilmente un overhead spaziale e di banda. Per ogni blocco dati richiesto, il processore è costretto a recuperare dalla lenta memoria esterna l'intera catena di *Counter Chunk* fino alla radice. Questo processo, se eseguito in modo ingenuo ad ogni operazione di memoria, saturerebbe rapidamente la larghezza di banda del bus, vanificando i vantaggi

prestazionali ottenuti dalla parallelizzazione. Per mitigare questo fenomeno e rendere l'architettura realmente scalabile, si rende necessaria l'introduzione di una **Last Level Cache (LLC)** centralizzata e direttamente accoppiata al nodo *Directory*. L'obiettivo di questa cache dedicata è memorizzare i blocchi dati e i nodi del TEC-Tree più frequentemente utilizzati. Poiché i rami superiori dell'albero (vicini alla radice) vengono condivisi da migliaia di foglie sottostanti, presentano un'altissima località spaziale e temporale. L'inserimento di una LLC permette quindi di "assorbire" la stragrande maggioranza delle richieste di autenticazione: se un *Counter Chunk* genitore è già presente in cache, il processore non deve accedere alla DRAM off-chip per recuperarlo. Questa scelta architetturale funge da ammortizzatore per il traffico di rete, permettendo al controllore di ispezionare i metadati crittografici quasi istantaneamente e riducendo al minimo gli accessi onerosi alla memoria principale. L'implementazione concreta di questa specifica ottimizzazione architetturale, ovvero l'integrazione del TEC-Tree supportato da una LLC centralizzata all'interno di un protocollo di coerenza, costituisce l'oggetto centrale e il contributo principale della presente tesi. Per modellare, simulare e validare una gerarchia di memoria così complessa, si è reso necessario l'utilizzo di un ambiente di simulazione architetturale avanzato.

1.4 Ambiente di Simulazione GEM5

Il simulatore gem5 è una piattaforma modulare basata su eventi discreti utilizzata nell'ambito dell'architettura dei calcolatori. La composizione modulare del sistema offre la possibilità di riconfigurare, estendere o sostituire ciascun componente dell'architettura adattandosi a molteplici esigenze di simulazione. Tale tool permette di emulare diversi sistemi di calcolo, permettendo la simulazione di diverse architetture come *Alpha*, *ARM*, *MIPS*, *Power*, *SPARC*, *RISC-V* e *x86* a 64 bit. Il simulatore è scritto principalmente in linguaggio C++ e Python e offre spiccata flessibilità nel costruire sistemi di memoria grazie a cache e crossbar ma soprattutto grazie al modulo Ruby, utile per una modellazione avanzata. Le modalità di simulazione previste da gem5 sono principalmente due: **full system mode**, simulando dispositivi e sistemi operativi completi o **syscall emulation** per simulazioni circoscritte nello spazio utente.

1.4.1 Modulo Ruby

Il nucleo centrale di questa ricerca si focalizza sull'impiego del modulo di simulazione Ruby, uno strumento progettato per simulare in modo estremamente accurato il comportamento delle interconnessioni e dei sottosistemi di memoria nei processori moderni. Grazie alla sua spiccata flessibilità nel modellare le gerarchie di memoria e i relativi protocolli, Ruby costituisce l'ambiente ideale per la progettazione

e il testing di nuove soluzioni. In questo contesto, il modulo verrà impiegato per concretizzare il protocollo di sicurezza ed integrità proposto, l'implementazione del quale costituisce il contributo principale del presente lavoro.

1.5 Obbiettivi e Workflow della Tesi



Figura 1.1: Workflow del processo.

Il workflow è articolato in quattro fasi e prevede:

- Analisi dei Fondamenti Teorici su TEC-TREE: in questa fase verranno approfondite le specifiche del protocollo di Autenticazione della memoria.
- Progettazione del Protocollo: sulla base dei fondamenti teorici si procederà con la progettazione del protocollo mediante la definizione di una macchina a stati finiti, messaggi ed eventi.
- Implementazione del Protocollo: gli elementi progettati nella fase precedente saranno tradotti nella sintassi del linguaggio del simulatore gem5 e opportunamente integrati con il protocollo di coerenza preesistente.
- Validazione e Analisi Prestazionale: si procederà alla modellazione e configurazione di un'architettura multicore all'interno del-

l'ambiente di simulazione, al fine di eseguire benchmark multithread. Tali scenari di test permetteranno di analizzare differenti casi d'uso e di condurre uno studio quantitativo sull'impatto prestazionale, sull'efficacia e sulla scalabilità del protocollo di sicurezza sviluppato.

Bibliografia

- [1] Emanuele Di Maio. Sviluppo di un protocollo di coerenza di cache expiration-based nel simulatore gem5. Master's thesis, Università degli Studi di Napoli Federico II, Napoli, Italia, 2025. Corso di Laurea Magistrale in Ingegneria Informatica, Relatore: Prof. Alessandro Cilardo.

- [2] Xiangyao Yu, Hongzhe Liu, Ethan Zou, and Srinivas Devadas. Tardis 2.0: Optimized time traveling coherence for relaxed consistency models. In *2016 International Conference on Parallel Architecture and Compilation Techniques (PACT)*, pages 241–254. IEEE, 2016.